

A. Abstract

This artifact contains a compact implementation of the Loop-Decoupled Prefetcher (LDP) on GEM5 [11], AArch64 workload binaries, inputs, compatible checkpoints, and scripts for running and validating experiments. It evaluates six linked-data-structure applications in nine task/input combinations. The workflow reproduces a representative speedup subset of Fig. 18 and a mechanism-level loop-decoupling ablation. It provides a bounded *fast* profile for timely evaluation and a *full* profile that executes each workload to completion and more closely reflects the paper’s full-execution trend.

B. Artifact check-list (meta-information)

- **Programs:** GEM5; Breadth-First Search (BFS), Single-Source Shortest Path (SSSP), and Minimum Spanning Tree (MST) from the Boost Graph Library [19] (BGL); Hash Join Probe (HJ-P) and Group-by (GB) [17] in database; and IPv4 lookup.
- **Compilation:** SCons; GCC; Python 3.10; a prebuilt GEM5 binary is also supplied
- **Data set:** cage10 and sx-superuser edge lists; database inputs are generated deterministically by the binaries.
- **Run-time environment:** x86-64 Linux; Docker Engine is recommended; no root access is required after Docker installation.
- **Hardware:** minimum 4 CPU cores, 16 GB RAM, and 15 GB free disk.
- **Execution:** nine no-prefetch/no-loop/full-LDP GEM5 comparisons, parallelized by a Python runner; supplied checkpoints are read-only.
- **Metrics:** GEM5 *simSeconds*, prefetch coverage, and timeliness; speedup is $T_{\text{noPF}}/T_{\text{LDP}}$.
- **Output:** stats and logs, speedup and mechanism summaries, and the loop-decoupling ablation in Fig. 25; automatic validators check expected results.
- **Disk/time:** the source archive is about 18 MB and the compressed checkpoints are about 88 MB; the preserved OCI image is about 796 MB, and build/output space is below 10 GB. Local validation with 12 workers takes approximately 11 minutes for the fast profile and 28 minutes for the full profile.
- **Availability/licensing:** public GitHub and GHCR releases; author-developed LDP-gem5 modifications are BSD-3-Clause, while bundled upstream software and data retain the licenses and notices documented in `THIRD_PARTY.md`.
- **Archived:** Zenodo version DOI: <https://doi.org/10.5281/zenodo.21502244>.

C. Description

1) *How to access:* The repository is <https://github.com/Zongpc/LDP-gem5>. The evaluated source is tagged with the label `micro26-final-r1`. The evaluated image tag is `ghcr.io/zongpc/ldp-gem5:micro26-final`. The Zenodo deposit also includes the OCI image and checkpoint bundle.

2) *Hardware dependencies:* The artifact simulates an ARM target and is insensitive to the host microarchitecture. We recommend at least four x86-64 cores, 16 GB RAM, and 15 GB free disk.

3) *Software dependencies:* The recommended path requires only Docker Engine. The native path was tested on Ubuntu 24.04 and requires GCC, SCons, Python 3.10 development files, zlib, M4, and pkg-config. All workload binaries and

inputs are included. `THIRD_PARTY.md` records provenance, licenses, citations, transformations, and SHA-256 hashes.

D. Installation

For Docker, pull the fixed image:

```
IMAGE=ghcr.io/zongpc/ldp-gem5:micro26-final
docker pull "$IMAGE"
```

For a native build, install the packages listed in `ARTIFACT.md`, then run the documented command. The Docker path is preferred because it pins all build and run-time dependencies.

E. Experiment workflow

The following commands run all tasks without a host bind mount and then export the results:

```
docker create --name ldp-ae-fast \
  --platform linux/amd64 "$IMAGE" \
  --run-profile fast --jobs 4
OUTPUT=ldp-results-fast
docker start -a ldp-ae-fast && \
  mkdir "$OUTPUT" && \
  docker cp ldp-ae-fast:/results/. "$OUTPUT/" && \
  docker rm ldp-ae-fast
```

where `IMAGE` is the fixed GHCR name above. Repository launchers provide the same sequence for Linux/macOS/WSL and Windows PowerShell. For each task, the runner restores the same checkpoint for no-prefetch, no-loop, and full-LDP configurations. The fast profile executes 10 million instructions per task except GB at 60 million; replacing *fast* with *full* runs all tasks to normal completion. It writes commands, stats, and logs, computes and validates the speedup and mechanism summaries, and generates `results/fast/analysis/mechanism.png`.

F. Evaluation and expected results

The paper evaluates applications using a broader workload suite. For portable deployment, this artifact selects six applications from Fig. 18, uses two compact graph inputs, medium-sized database inputs, and IPv4-m, and provides fast and full run profiles. All three configurations restore the same checkpoint and execute effectively equal instruction counts within a profile. The prebuilt simulator, checkpoints, and Docker environment remove host-dependent setup differences. In local validation with 12 workers, the fast and full profiles complete in approximately 11 and 28 minutes, respectively.

The fast profile is intended for timely functional and trend validation, while the completion-based profile produces stronger improvements and follows the paper’s full-execution trend more closely. The one-command runner performs speedup validation as its first evaluation substep and loop-decoupling-ablation validation as its second. After both validators succeed, the workflow displays a prominent final banner containing `CHECK PASSED` and `LOOP DECOUPLING IS EFFECTIVE`, together with the observed overall no-loop and full-LDP speedups.

G. Mechanism-level ablation

The default workflow compares full LDP with a variant that disables loop decoupling and validates the resulting mechanism summary as a separate evaluation substep. Both profiles produce `mechanism.csv`, `mechanism_summary.csv`, and `mechanism.png` for the loop-decoupling ablation in Fig. 25.

H. Notes

The repository README documents additional simulation controls, including task selection, parallelism, checkpoint and output directories, CPU profiles, and instruction limits. These options allow reviewers to run focused checks or explore configurations beyond the default automated workflow.

An earlier bind-mount command could fail when the image UID/GID differed from the host user, and a POSIX UID mapping is not portable to Windows Docker Desktop. The default create/start/copy workflow addresses this issue by writing to the image-owned `/results` and exporting to a newly created host directory after execution; it therefore requires neither a writable bind mount nor host UID/GID mapping.

For IPv4, timeliness without loop decoupling is nearly 100% because the AE configuration recognizes only the outermost streaming accesses: the few issued prefetches are timely, but their coverage and performance benefit are negligible. Loop decoupling exposes the complete access relationship while retaining high LDS prefetch timeliness and providing substantial acceleration.